



VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

Rendelőtablet

<https://github.com/szoft2006A1Cs6/Vizsgaremek>

Készítették:

DÉNES ZOLTÁN

FARKAS LÁSZLÓ

HORVÁTH BENDEGÚZ

SZOMBATHELY

2026

I. Bevezetés

A projektünk egy asztali, tablet alapú rendszer, ami teljesen megváltoztatja az éttermi rendelés menetét. A vendégek közvetlenül az asztaloknál böngészhetik az étlapot, és egy érintéssel le is adhatják a választásokat, ami azonnal a konyhára kerül. Ez a megoldás kiküszöböli a felesleges várakozást, és leveszi a terhet a felszolgálók válláról, hiszen minden információ digitálisan, hiba nélkül jut el a megfelelő helyre. Olyan felületet alkottunk, ami a modern technológiát használja arra, hogy a vendéglátást egyszerűbbé, gyorsabbá és élvezhetőbbé tegye.

II. Szoftver célja

A szoftver fejlesztésénél több célt is megfogalmaztunk. Egyrészt elengedhetetlenek voltak az olyan alapvető szempontok, mint az adatvédelem és a megbízhatóság, másrészt természetesen a bevételszerzés is kiemelt célunk volt. A legfontosabb prioritásunk a biztonságos rendszer kialakítása volt, mind a fizetés, mind az adatvédelem terén. Kiemelten fontos számunkra, hogy a vendégek személyes adatai ne kerülhessenek illetéktelen kezekbe, hiszen egy adatvédelmi hiba a szoftver hírnevét is jelentősen rontaná. Nagy hangsúlyt fektettünk a jogosultságkezelésre is: jelszóval védett admin felületet hoztunk létre, hogy elkerüljük az esetleges visszaéléseket, és megakadályozzuk, hogy a vendégek a rendszer adminisztrációs részéhez hozzáférhessenek. Bár ezek a biztonsági fejlesztések közvetlen profitot nem termelnek, elengedhetetlenek a rendszer megbízható működéséhez.

A rendszerrel egy gyakori problémára is megoldást kínálunk: a pincérrel való kommunikáció során előforduló félreértéseket. A tabletes rendelésnél a vendég saját maga választja ki az ételeket, a végén pedig áttekintheti a listát, így biztos lehet benne, hogy valóban azt rendelte, amit szeretett volna. Mivel nem kell a pincérre várni, aki egyszerre csak egy rendelést tud felvenni, a tablet segítségével egyszerre több asztal is leadhatja az igényét, ezzel kiküszöbölve a várakozást és a bonyolultabb folyamatokat. A vendég nyugodtan átgondolhatja vagy könnyen módosíthatja a rendelését, a tablet használatával pedig minimálisra csökken a személyzettel folytatott közvetlen kommunikáció szükségessége, ami sokak számára kényelmesebb élményt jelent. Emellett a vendég a tablet kijelzőjén folyamatosan követheti a fogyasztása végösszegét, így könnyebben a pénztárcájához

igazíthatja a választását, szemben a hagyományos rendelésnél megszokott, gyakran bizonytalan végösszeggel.

A technológia fejlődésével ez a megoldás a pincérek munkáját is kiegészítheti, lehetőséget adva az étteremnek, hogy a felszolgálókat más területeken foglalkoztathatják. A rendszer a környezetvédelmet is támogatja: a digitális étlappal kiválthatjuk a papíralapú menüket, így elkerülhető az étlapok folyamatos újranyomtatása minden árváltozás vagy termékcseré esetén. Magyarországon a digitális rendelés a gyorséttermeken kívül még újdonságnak számít, így ez a fejlesztés vonzó lehet azoknak a vendégeknek, akik nyitottak az új technológiákra. Célunk továbbá a vásárlói elégedettség azonnali mérése egy értékelési funkcióval.

III. Célközönség:

Alapvetően az általunk készített projektet azon éttermekhez szeretnénk eljuttatni, akik fejlődést/újítást szeretnének hozni az éttermükben a technológia segítségével vagy akár csak szimplán szeretnék felvenni a versenyt a gyorséttermi rendeléssel szemben. Emellett az is szóba jöhet, hogy az alkalmazottak munkáját szeretnék egyszerűsíteni, akár gyorsabbá tenni, ez az oldal nekik is nagyszerű választás.

IV. Feladatok:

Feladat:	Készítette:
Adatbázis / Entity Relationship Diagram	Dénes Zoltán, Farkas László
ASP.NET Web Application / Unit Test (Backend)	Dénes Zoltán, Horváth Bendegúz
Frontend	Farkas László, Horváth Bendegúz
Dokumentáció	Dénes Zoltán, Farkas László, Horváth Bendegúz

V. Komponenteinek technikai leírása

A Gusto Bistro rendszer egy modern, három rétegből felépülő rendszer, amely biztosítja az adatok biztonságos tárolását, a hatékony üzleti logika futtatását és a responszív felhasználói élményt. Ez a felépítés lehetővé teszi a komponensek egymástól független fejlesztését és skálázását.

Frontend:

A felhasználói felület egy **React-alapú egyoldalas alkalmazás**. Ez a réteg felelős a vendégek és a személyzet vizuális kiszolgálásáért.

- **Komponens-alapú felépítés:** A felület moduláris, újrahasználatos komponensekből áll (pl. étlap-kártya, kosár-elem, értesítési sáv). Ez gyorsabb fejlesztést és egységes megjelenést tesz lehetővé.
- **Állapotkezelés (State Management):** A kliensoldali állapotot (pl. a kosár tartalma vagy a bejelentkezett felhasználó adatai) a React saját state-kezelése és Context API-ja felügyeli.
- **Adatkommunikáció:** A frontend aszinkron módon kommunikál a backenddel. Ez biztosítja, hogy a rendelés leadása vagy a pincérhívás ne okozzon oldalújrátöltést, így a felhasználói élmény folyamatos marad.

Backend:

A projekt alapja a C# nyelven írt **ASP.NET Core Web API**. Ez a réteg közvetíti a felhasználói felület és az adatbázis között.

- **RESTful architektúra:** A backend egy REST API-t valósít meg, ahol az erőforrásokat (asztalok, ételek stb.) szabványos HTTP metódusokon keresztül érjük el.

- **Üzleti logika és Biztonság:** Itt történik a bemeneti adatok validálása és a hitelesítés. A rendszer **JWT** alapú autentikációt használ. Ez biztosítja, hogy a vendégek ne férhessenek hozzá a konyhai funkciókhoz, és fordítva.
- **Dependency Injection:** A .NET beépített Dependency Injection konténerét használtuk a kontrollerek és az adatbázis-kontextus közötti kapcsolat megteremtéséhez.

Adatbázis:

A rendszer alapja egy relációs adatbázis-kezelő rendszer, jelen esetben a **MySQL**, amelyet a fejlesztési fázisban az **XAMPP** környezet biztosít. Az adatbázis felelős a termékek, rendelések, asztalok és felhasználói adatok tartós tárolásáért.

- **Technológia és ORM:** A backend és az adatbázis közötti kapcsolatot az Entity Framework Core (EF Core) biztosítja. Ez egy objektum-relációs leképző (ORM) eszköz, amely lehetővé teszi, hogy C# osztályokon keresztül kezeljük az adatbázis-műveleteket SQL lekérdezések kézi írása helyett.
- **Adatmodell tervezés:** Az adatbázis normalizált formában tárolja az adatokat a redundancia elkerülése érdekében. A táblák közötti kapcsolatokat idegen kulcsok biztosítják (pl. a rendelestétel tábla hivatkozik a termék és a rendeles táblákra).

VI. Működésének műszaki feltételei

Ahhoz, hogy egy tablet alapú rendelési rendszer megbízhatóan és gyorsan működjön az étteremben, a háttérben technikai alapokra van szükség.

1. Hardveres eszközök

Tabletek (kliensek): Az asztalokon elhelyezett eszközök, amelyek megfelelő töltőegységgel és rögzített tokkal rendelkeznek.

Központi egység (szerver): Egy helyi számítógép vagy felhő alapú szerver, amely fogadja a rendeléseket, kezeli az adatbázist és szinkronizálja az információkat.

Konyhai megjelenítő: A konyhán elhelyezett monitor vagy tablet, amely a beérkező rendeléseket azonnal megjeleníti vagy kinyomtatja a séfek számára.

2. Hálózati infrastruktúra

Stabil Wi-Fi hálózat: Kiváló minőségű routerekre van szükség, amelyek kezelni tudják a több egyidejű kapcsolatot (asztalok száma + konyhai eszközök) anélkül, hogy a kapcsolat megszakadna vagy lassulna.

Hálózati biztonság: A rendszer elkülönítése az éttermi vendég Wi-Fi-től, hogy illetéktelenek ne férhessenek hozzá a rendelési adatokhoz.

VII. Technológiai háttér:

Backend: C#

Frontend: [React.js](#) (router dom), Css,

Backend - Frontend kapcsolat: REST API

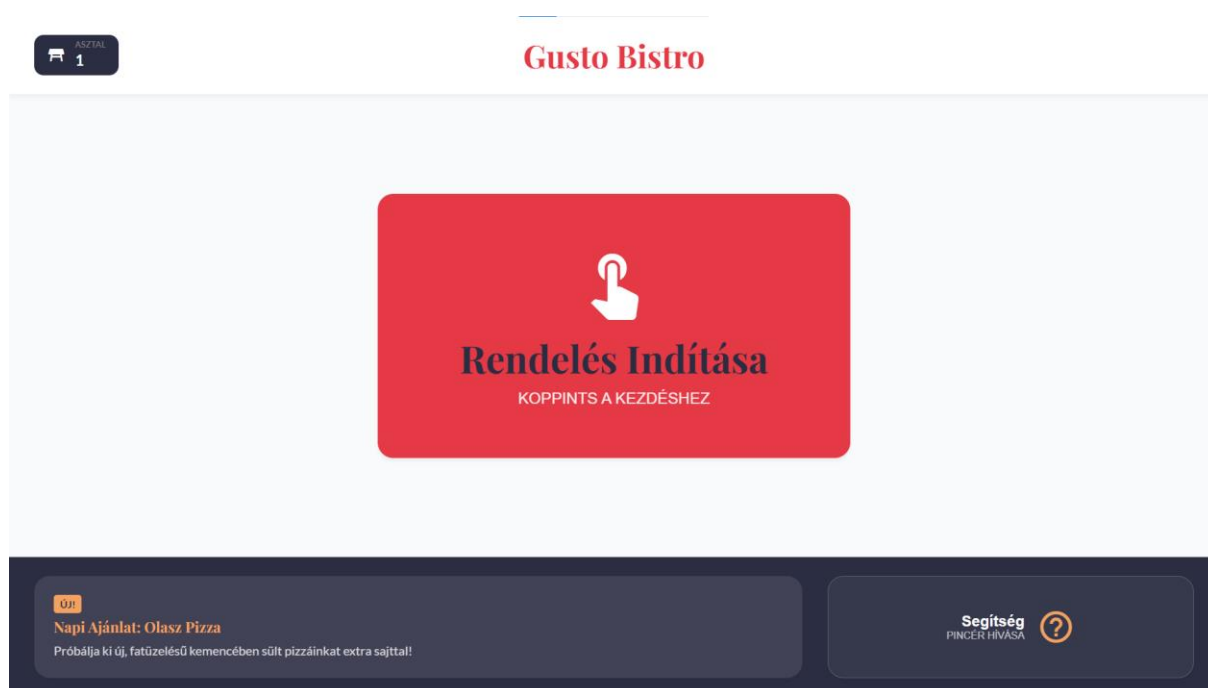
Adatbázis: XAMPP MySql - 3306 port

Adatformátum: JSON

VIII. Használatának rövid bemutatása:

Vendég mód:

Az asztali felhasználóba belépve az alábbi képernyőt látjuk:



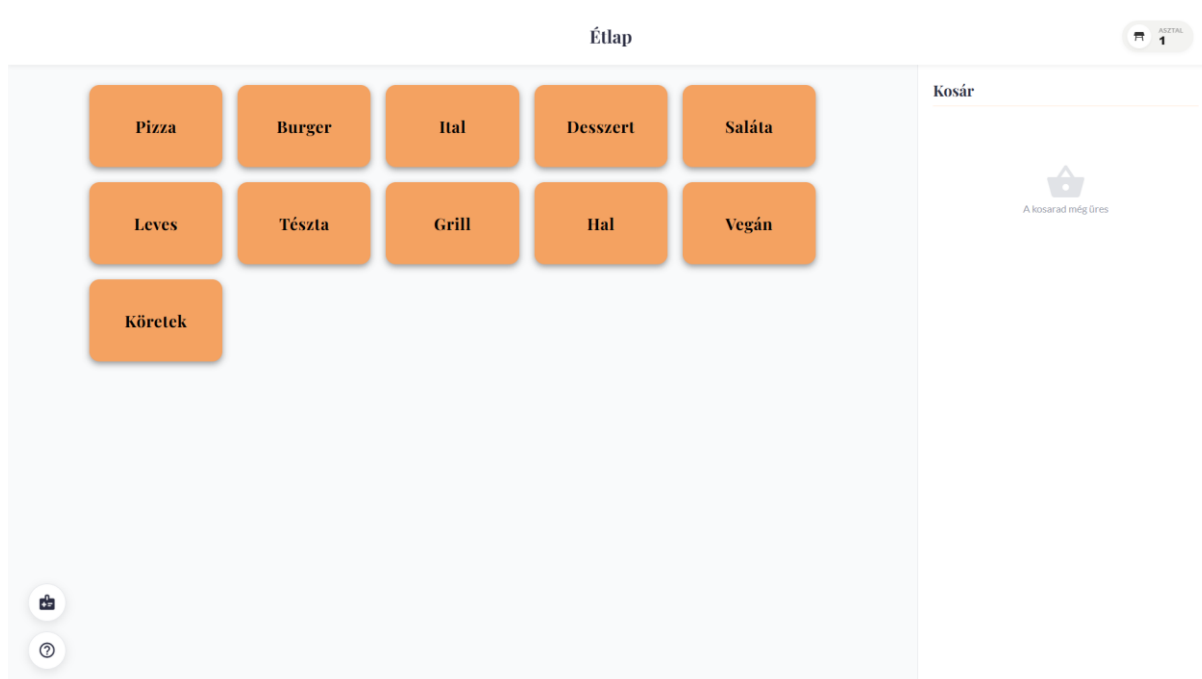
1. Ábra Kezdőoldal

A bal felső sarokban található az adott asztal száma.

Az alsó menüsorban az adott “napi ajánlatot” találjuk. Amennyiben a vendég kiválasztja a napi ajánlatot, a termék automatikusan a kosárba kerül.

A “segítség gombra” kattintva pincér hívást küld a konyhai oldalnak.

A “rendelés indítása” gombra kattintva a vendég a menü oldalra kerül:



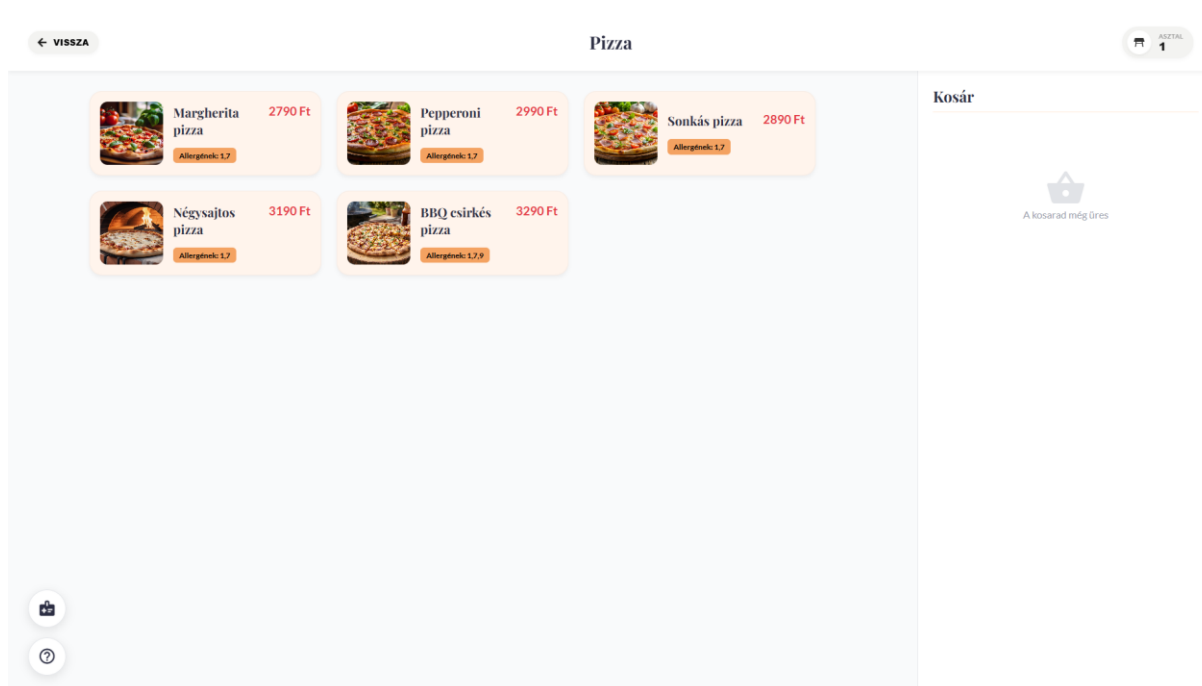
2. Ábra Étlap

A bal alsó sarokban található gombok a vásárló segítségére szolgálnak.

A kérdőjellel jelölt gomb (pincérhívó) funkciója megegyezik a főoldalon lévővel.

A felső gomb listázza az allergének számát a leírásával társítva.

Kategória választás után megjelenítésre kerülnek az ahhoz tartozó termékek.



3. Ábra Termékválasztás

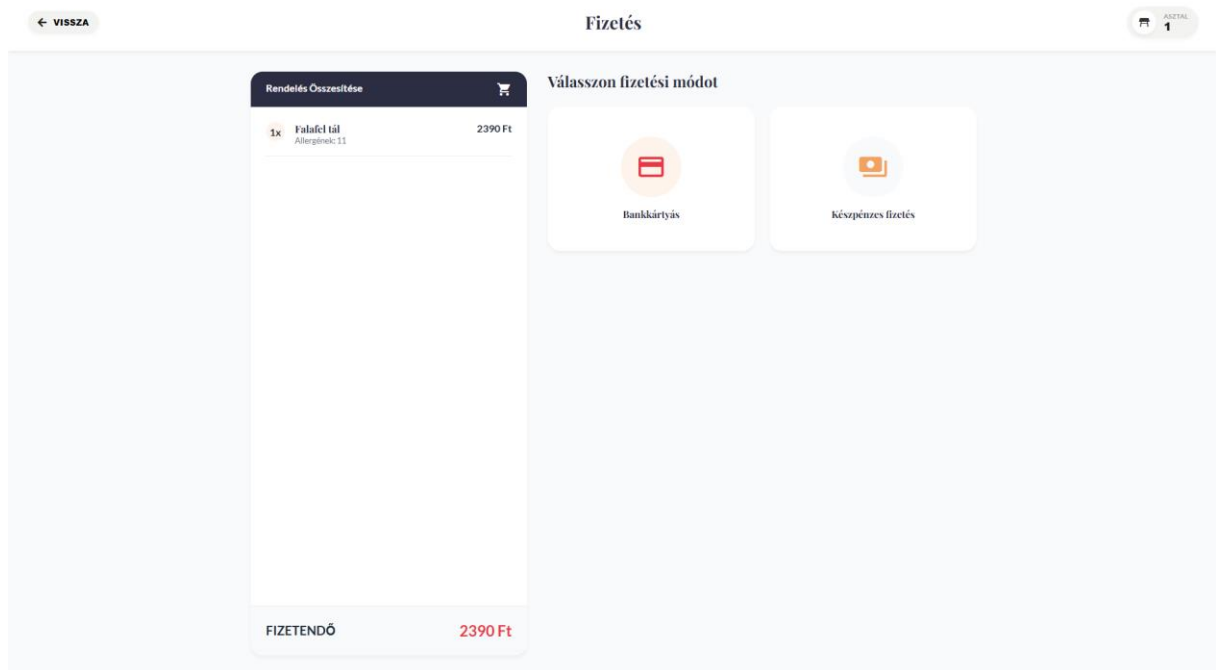
A kiválasztott termékek a “kosárba” kerülnek amelyet a képernyő jobb oldalán találhatunk.

A kosárban lévő termékek mennyiségét tudjuk növelni, illetve eltávolítani.

Megjelenítésre kerül a termékek ára és a végösszeg is.

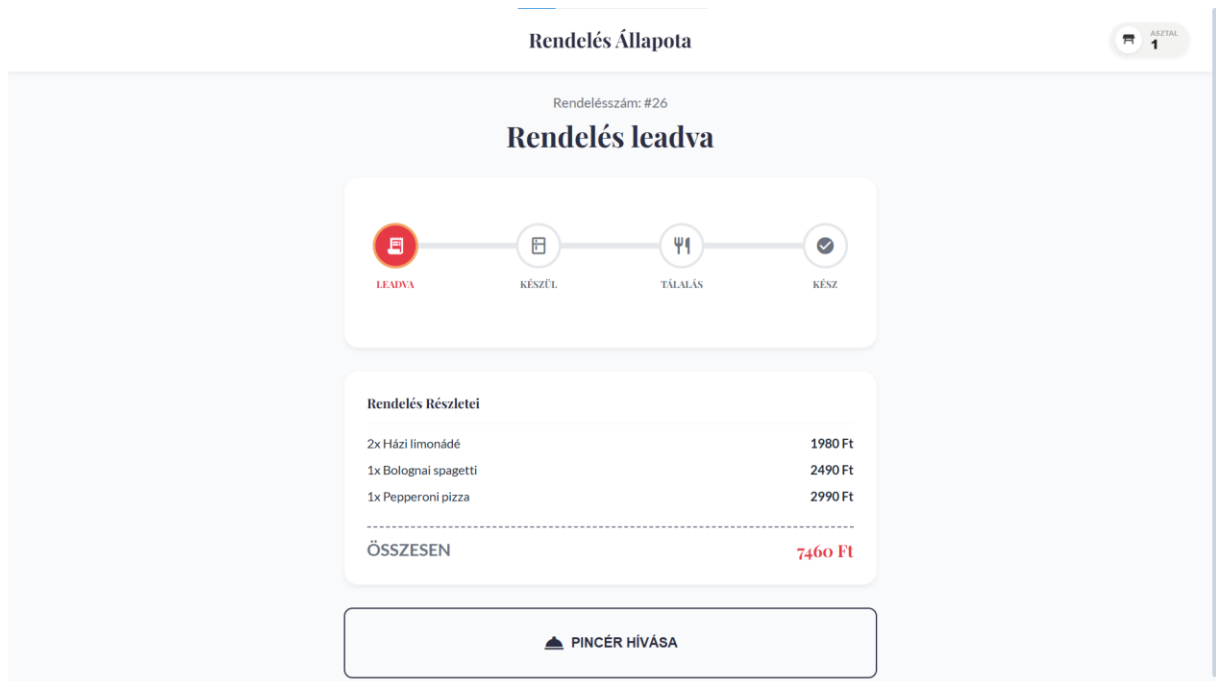
A fizetés gombra kattintva továbbdob az összesítő oldalra.

Ezen az oldalon ki tudjuk választani, hogy milyen módon szeretnénk fizetni. Mindkét esetben a konyha értesítést kap a fizetési szándékról.



4. Ábra Fizetés

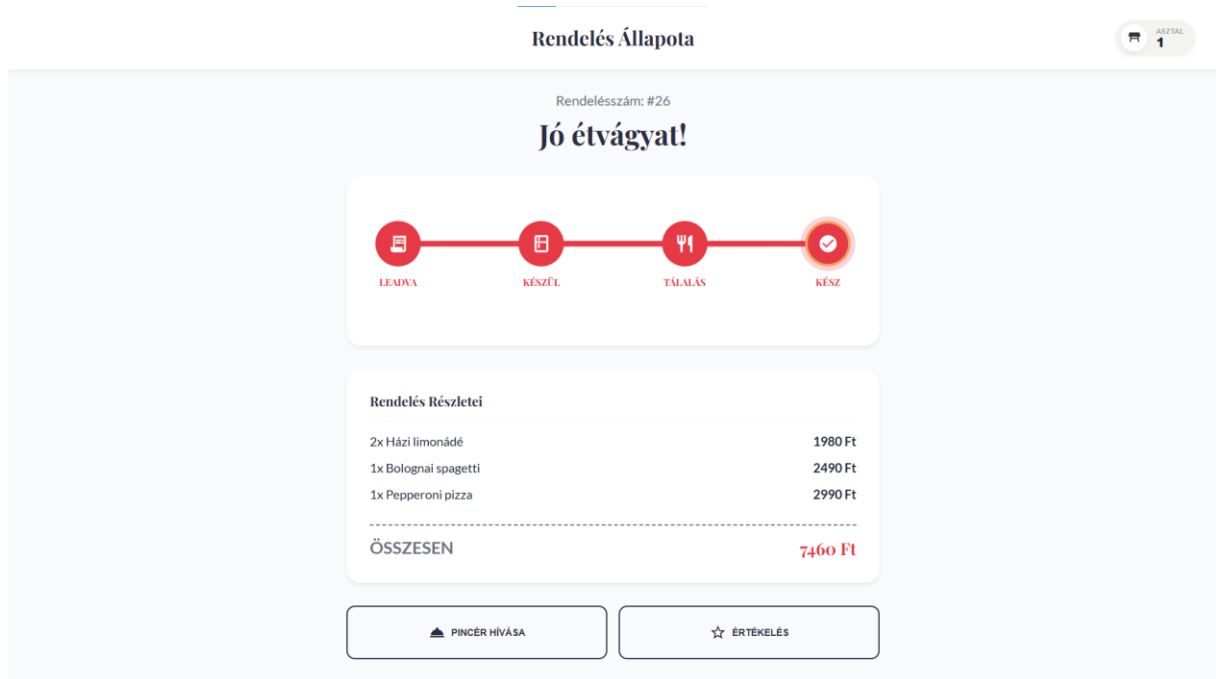
A vásárlás befejezése után, a vendégnek lehetősége van, hogy nyomonkövesse rendelése állapotát és megtekintse a rendelés részleteit.



5. Ábra Állapot

Probléma esetén a “pincér hívás” gombra kattintva segítséget tud kérni.

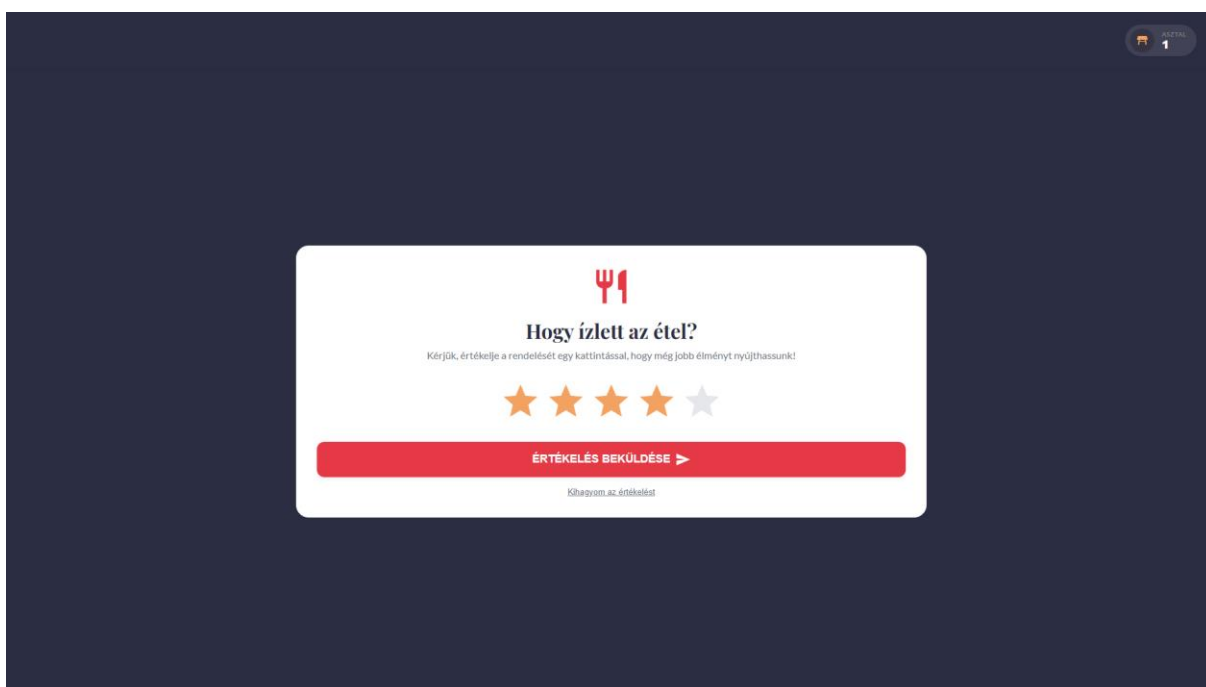
A rendelés folyamatát valós időben nyomonkövetheti.



6. Ábra Étel kész

Az étel elkészülését követően, a vendég tetszése szerint bármikor értékelhet.

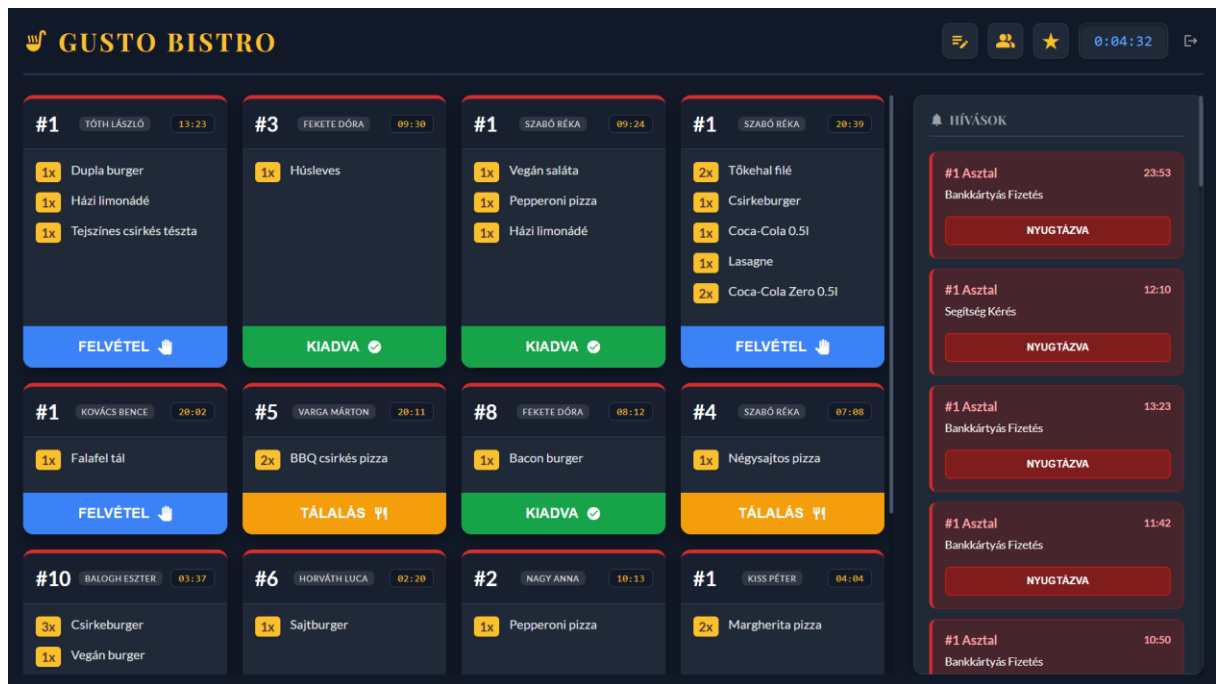
A vendégnek lehetősége van kihagyni az értékelés.



7. Ábra Értékelés

Konyhai mód:

A konyhai oldalba való belépés után az adott képernyőt láthatjuk.



8. Ábra Konyhai felület

A különböző rendelés kártyákon megtalálható az asztal száma, a pincérnek a neve illetve a rendelés leadásának időpontja.

Továbbá megtalálhatóak a rendelés tételei.

A jobb felső sarokban a jelenlegi idő, egy editmenu, egy editstaff és egy ratingstats gomb található, illetve visszalépés a felhasználóváltó felületre.

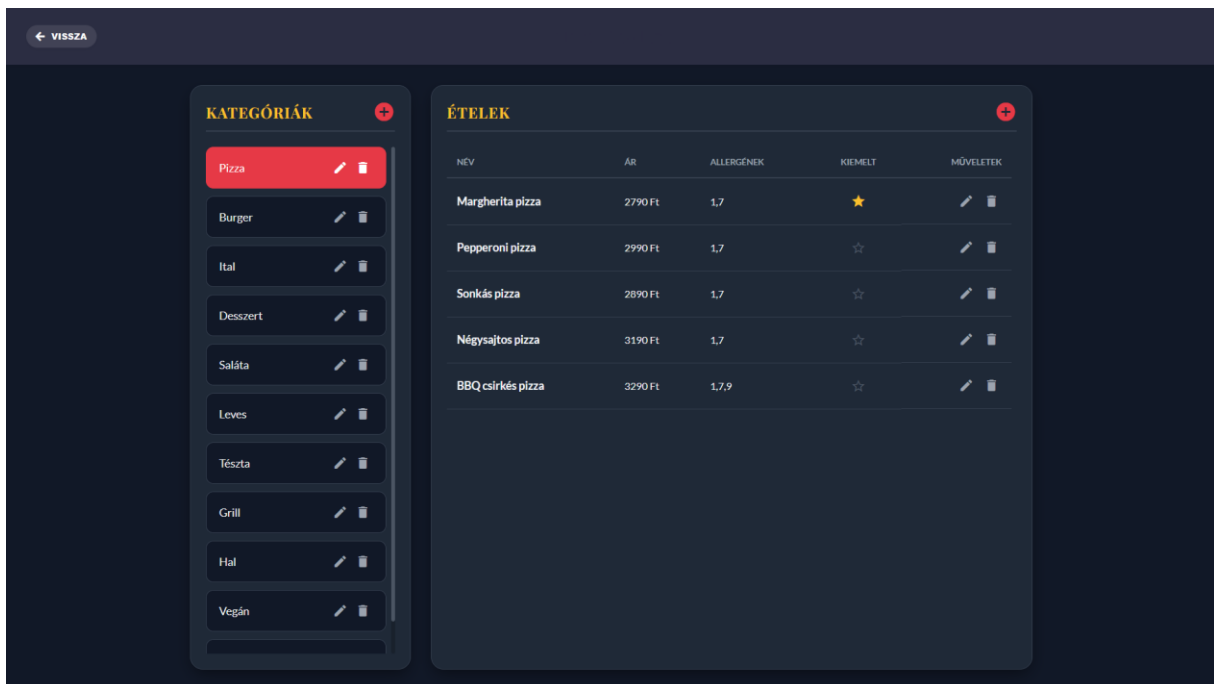
Az alsó sávban 3 állapot található meg.:

Felvétel - A gombra kattintva a rendelés folyamata “Készül” stádiumba lép.

Tálalás - Kattintásra a rendelés “Tálalás” állapotúra változik.

Kiadva - Miután megkapta a vendég a rendelését a dolgozók erre a gombra kattintva lezárják a rendelést, ezért a konyhai oldalon eltűnik, a vendégnél pedig a “Kész” állapot jelenik meg.

A felület jobb oldalán található, hogy melyik asztal kérelmezett fizetést, illetve melyik asztalnak van szüksége pincéri segítségre. A “nyugtázva” gombra kattintva eltűnik a kérelem.



9. Ábra Étel módosítás

Az editmenu gomb lenyomás után ezt a felületet kapjuk meg.

Ezen felületen jelennek meg a termékek különböző kategóriákra osztva.

A toll ikonra kattintva a kategóriák nevét tudjuk változtatni.

A kuka ikonra kattintva tudjuk törölni az adott kategóriát.

Az kategóriák részen a jobb felső sarokban található egy plusz gomb, ahol egy teljesen új ételt/italt tudunk hozzáadni az adott kategórián belül.

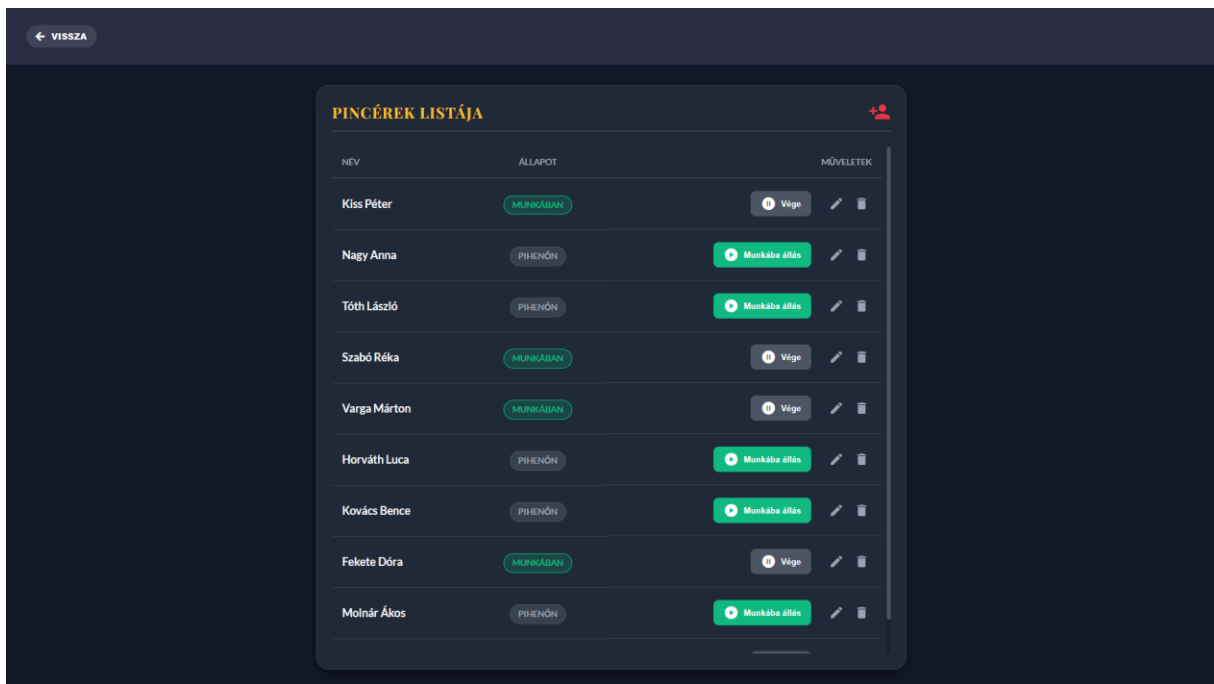
Az egyes kategóriák kiválasztása után megjelennek az oda tartozó ételek/italok.

Ahol a toll ikonra kattintva tudjuk szerkeszteni az ételek nevét, árát, allergénjeit illetve napi ajánlatnak tudjuk őket beállítani.

Ugyanitt tudjuk a képeknek a linkjeit beszúrni, illetve változtatni.

A kuka ikonra kattintva tudjuk törölni a megadott ételt/italt.

Az ételek részen a jobb felső sarokban található egy plusz gomb, ahol egy teljesen új ételt/italt tudunk hozzáadni az adott kategórián belül.



10. Ábra Pincérek

Az editstaff gomb lenyomása után ezt a felületet kapjuk meg.

Itt láthatóak az étteremben dolgozó pincérek nevei.

Az állapot jelzésnél kapunk arról visszajelzést, hogy az adott pincér munkában van-e vagy pedig pihenőn (a pihenő kategóriába esik természetesen az adott napon nem dolgozó személy is).

A zöld munkába állok gomb lenyomásával tud egy pincér munkába állni, ezek után eshet rá rendelés.

Ha nincs pincér munkában, akkor a fő konyhai oldalon “Nincs Pincér” piros felirat jelenik meg a pincér neve helyett a leadott rendelés kártyáján.

A toll ikonra kattintva tudjuk a pincér nevét állítani.

A kuka ikonra nyomva tudjuk az adott pincért kirúgni.

A jobb felső sarokban található személyfelvétel gomb lenyomásával tudunk új pincért felvenni az étteremben.

← VISSZA

BEÉRKEZETT ÉRTÉKELÉSEK

DÁTUM	RENDELÉS ID	PONTSZÁM	MŰVELETEK
2026. 04. 22. 12:11	#28	3 ★	
2026. 04. 21. 11:50	#26	4 ★	
2026. 04. 21. 10:51	#25	5 ★	
2026. 04. 21. 10:38	#23	5 ★	
2026. 04. 21. 10:32	#22	4 ★	
2026. 04. 21. 09:02	#18	3 ★	
2026. 04. 20. 20:44	#16	5 ★	
2025. 11. 02. 00:00	#1	5 ★	
2025. 11. 02. 00:00	#2	4 ★	
2025. 11. 02. 00:00	#3	5 ★	

11. Ábra Értékelések

Az ratingstats gomb lenyomása után ezt a felületet kapjuk meg.

Itt találhatóak a vendégek által küldött visszajelzések.

A visszajelzések időpontja.

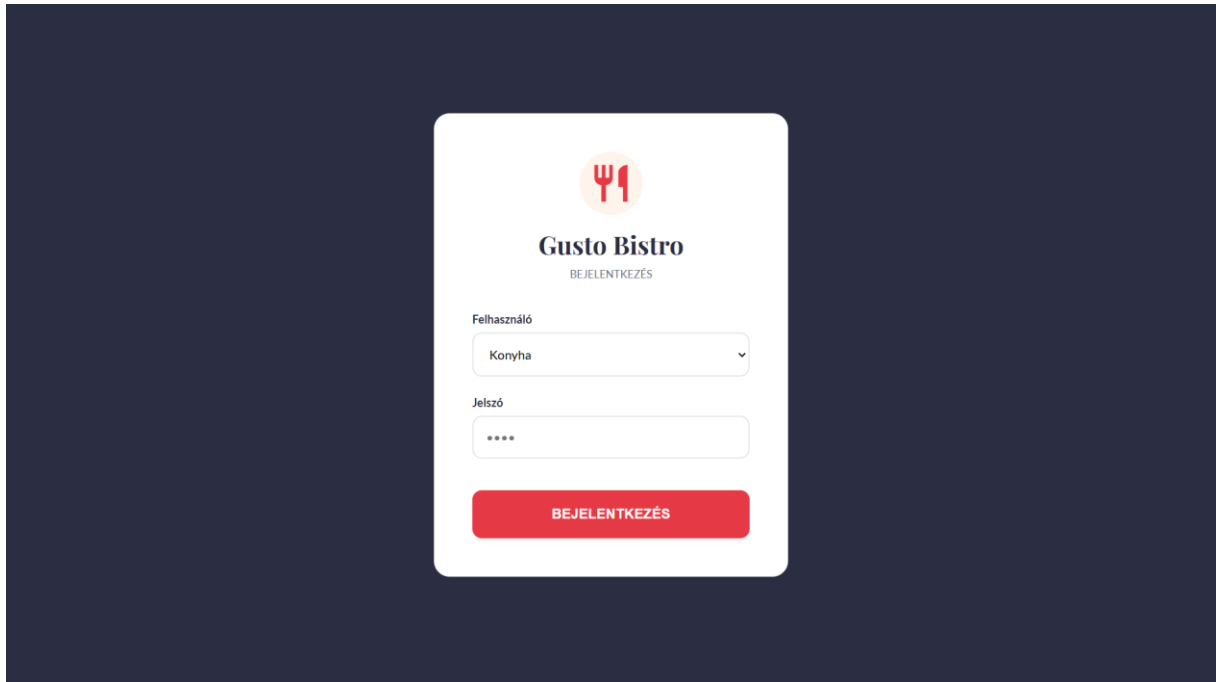
Magának a rendelésnek az ID-ja.

A vendég által megadott pontszám.

És az ezen a felületen látható kuka ikonnal tudunk ezek között az értékelések között törölni.

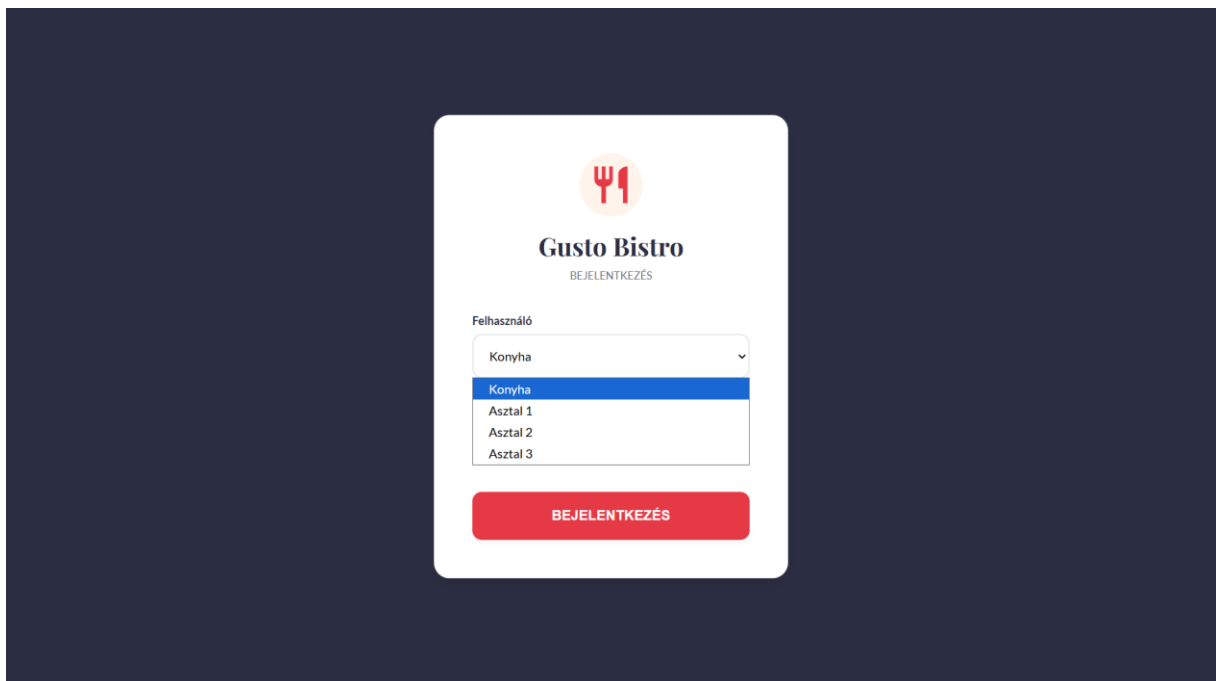
Bejelentkezés

A bejelentkezési oldal kinézete:



12. Ábra Bejelentkezés

Az alábbi módon kiválasztható a kívánt felhasználó:



13. Ábra Bejelentkezés II.

Minden felhasználónak azonos a jelszava, jelen esetben '1234' ez a kódban módosítható. Ehhez a felülethez a vendégeknek nincs hozzáférésük. Tehát ők így kárt nem tudnak okozni a konyhai oldalon.

IX. Adatbázis terv (ER modell) [Melléklet: 14. ábra]:

Táblák és mezők:

A Gusto Bistro rendszer adatbázisa a rendelőtablet nevet viseli, és MySQL környezetben fut. Az adatbázis 8 jól strukturált táblát tartalmaz, melyek biztosítják a relációs adatok (asztalok, ételek, rendelések és visszajelzések) tartós tárolását. Az alábbiakban bemutatjuk a táblák pontos szerkezetét és a mezők technikai specifikációját.

1. asztal tábla

Az étterem fizikai vendégtereinek kapacitását tároló entitás.

- asztal_id : Egyedi azonosító, amely megkülönbözteti az egyes asztalokat. Erre a mezőre hivatkoznak a rendelések és a pincérhívások.
- ferohely: Numerikus érték, amely meghatározza az adott asztal maximális befogadóképességét (pl. 2, 4, 6 vagy 8 fős asztalok).

2. pincer tábla

A felszolgálói személyzet adatait és aktuális státuszát nyilvántartó tábla.

- pincer_id : A dolgozó egyedi azonosítója.
- pincer_nev: A felszolgáló teljes neve (pl. "Kiss Péter").
- munka: Logikai érték, amely jelzi, hogy a pincér éppen aktív műszakban van-e, ennek az alapértelmezett értéke: 0.

3. etel_tipus tábla

Az étlap kategóriáit (pl. Pizza, Burger, Ital) tartalmazó törzsadat-tábla.

- eteltipus_id: A kategória egyedi azonosítója.

- `tipus_nev`: A kategória szöveges megnevezése, amely alapján a frontend menüje felépül.

4. termék tábla

Az étlap teljes kínálatát tároló központi adattábla.

- `termek_id`: A konkrét étel vagy ital egyedi azonosítója.
- `eteltipus_id`: Idegen kulcs, amely besorolja a terméket az `etel_tipus` táblában lévő kategóriák valamelyikébe.
- `termek_nev`: A termék pontos megnevezése (pl. "Margherita pizza").
- `ar`: A termék bruttó eladási ára forintban kifejezve.
- `allergenek`: Vesszővel elválasztott számok, melyek az ételben található allergéneket jelölik (pl. "1,7"). Üresen is hagyható.
- `featured`: Logikai érték (0 vagy 1), amely megmutatja, hogy a termék "napi ajánlat"-ként kiemelt helyen jelenjen-e meg a vendég tabletjén.
- `kep`: A termékhez tartozó kép (URL) elérési útja (Imgur hivatkozás).

5. rendelés tábla

A tranzakciók alapadatait (fejlécét) rögzítő entitás.

- `rendeles_id`: A megrendelés egyedi sorszáma.
- `pincer_id`: Idegen kulcs. Hivatkozás a felszolgálóra, aki a rendelést kiszolgálja.
- `asztal_id`: Idegen kulcs. Annak az asztalnak az azonosítója, ahonnan a rendelést leadták.
- `idopont`: A rendelés leadásának pontos dátuma és ideje.
- `statusz`: Számkódos állapotjelző, amely a megrendelés aktuális életciklusát jelöli (pl. 0: Leadva, 1: Készül, 2: Tálalás, 3: Kész).

6. rendelés_tétel tábla

Kapcsolótábla, amely részletezi, hogy egy adott rendelés pontosan milyen termékeket tartalmaz.

- `tétel_id`: A kosárba tett tétel egyedi sorszáma.
- `rendeles_id`: Idegen kulcs. Hivatkozás a fő rendelés táblára.

- *termek_id*: Idegen kulcs. Hivatkozás a kiválasztott termékre.
- *mennyiség*: Megmutatja, hogy a kiválasztott ételből/italból hány darabot tettek a kosárba.

7. **pincer_hivas** tábla

Az asztalokról érkező asszisztencia- és fizetési kérelmeket naplózó tábla.

- *hivas_id*: A hívás egyedi sorszáma.
- *asztal_id*: Idegen kulcs. Az asztal, ahonnan a kérés érkezett.
- *idopont*: A segítségkérés pontos másodperce.
- *statusz*: Szöveges állapot, ami leírja a kérés típusát (pl. "Segítség Kérés", "Bankkártyás Fizetés") vagy az állapotát ("Függőben", "Teljesítve").

8. **ertekeles** tábla

A minőségbiztosítási céllal leadott vendégvéleményeket raktározza.

- *ertek_id*: A visszajelzés egyedi azonosítója.
- *rendeles_id*: Idegen kulcs. Összekapcsolja az értékelést egy konkrét rendelési tranzakcióval.
- *idopont*: A vélemény leadásának időpontja.
- *pontszam*: A vendég által adott numerikus értékelés 1-től 5-ig számozott skálán.

Entitások Közötti Kapcsolatok (Idegen Kulcsok)

A relációs adatbázis adatintegritását a táblák közötti szigorú, egy-a-többhöz (1:N) kapcsolatok biztosítják az alábbiak szerint:

1. A **pincer_hivas** tábla *asztal_id*-ja hivatkozik az **asztal** tábla *asztal_id* mezőjére. (Egy asztal több hívást is leadhat).
2. A **rendeles** tábla *asztal_id*-ja hivatkozik az **asztal** tábla *asztal_id* mezőjére. (Egy asztalhoz több rendelés is tartozhat).

3. A **rendeles** tábla pincer_id-ja hivatkozik a **pincer** tábla pincer_id mezőjére. (Egy pincér több rendelést is kezelhet).
4. A **rendeles_tetel** tábla rendeles_id-ja hivatkozik a **rendeles** tábla rendeles_id mezőjére. (Egy rendelés több különböző tételt tartalmazhat).
5. A **rendeles_tetel** tábla termék_id-ja hivatkozik a **termek** tábla termék_id mezőjére. (Egy adott termék több rendelésben is szerepelhet).
6. A **termek** tábla eteltipus_id-ja hivatkozik az **etel_tipus** tábla eteltipus_id mezőjére. (Egy kategóriába több étel is tartozik).
7. Az **ertekeles** tábla rendeles_id-ja hivatkozik a **rendeles** tábla rendeles_id mezőjére. (Minden rendeléshez kapcsolható egy értékelés).

X. A Backend Kontrollerek Működése és Végpontjai

A rendszer logikáját kilenc, egymástól jól elkülöníthető controllerre bontottuk. Minden controller egy konkrét területért vagy adatbázis-entitásért felel, a hozzáféréseket pedig szigorú, JWT-alapú jogosultságkezelés védi.

1. AuthController – A rendszer biztonsági kapuja

Ez a controller felel az azonosításért és a jogosultságok kiosztásáért. Kiemelt szerepe van a hálózat védelmében.

- **Bejelentkezés (POST /api/Auth/login):** Nem tárolunk és nem is fogadunk el nyers jelszavakat. A beérkező adatokat a rendszer a **SHA256 algoritmussal** titkosítja (hash-eli), és ezt veti össze az adatbázissal. A jogosultságok kiosztása automatikus: ha a bejelentkező név tartalmazza az "Asztal" szót, a fiók **Guest (vendég)** jogkört kap, minden más esetben (pl. Konyha, Pincér) **Staff (személyzet)** jogkört. A válasz egy 8 óráig érvényes JWT token.
- **Felhasználók listázása (GET /api/Auth/users):** Ez egy publikus végpont, amely a frontend bejelentkező képernyőjének betöltését segíti. Kिलistázza a választható felhasználókat, hogy a tableteken a műszakkezdéskor csak egy gombra kelljen rábökölni.

2. AsztalController – Az éttermi térkép kezelése

A fizikai vendégtér adminisztrációjáért felelős modul.

- **Asztalok lekérése (GET /api/Asztal és GET /api/Asztal/{id}):** Itt kérdezi le a kliens, hogy hány asztal van és azoknak mekkora a kapacitása. Az id-alapú lekérdezésnél, ha nem létezik az asztal, a szerver elegánsan egy 404-es (NotFound) hibával tér vissza.
- **Asztalok karbantartása (POST és PUT):** Szigorúan csak személyzeti (Staff) joggal használható végpontok. Új asztalok felvételére vagy a meglévők férőhelyeinek módosítására szolgálnak (például ha egy asztalt kibővítenek 4 főre).

3. EtelTipusController – A digitális étlap kategóriái

Ez a kontroller strukturálja a kínálatot (pl. Levesek, Főételek, Italok).

- **Kategóriák listázása (GET):** Visszaadja a menüpontokat a bal oldali navigációs sávhoz.
- **Kiterjesztett lekérdezés (GET /{id}?ext=true):** Ez egy rendkívül hasznos funkció. Az Entity Framework Include metódusát (Eager Loading) használva, ha ezt a paramétert beküldjük, nemcsak a kategória nevét kapjuk meg, hanem beágyazva az összes hozzá tartozó ételt is. Ezzel rengeteg hálózati forgalmat spórolunk.

- **Adminisztráció (POST, PUT, DELETE):** Kategóriák létrehozása és módosítása. Érdekes, hogy a törlésnél a rendszer figyeli az adatintegritást: nem enged törölni olyan kategóriát, amiben még vannak aktív termékek.

4. TermekController – Az étlap konkrét tételei

Az étterem kínálatának fő része, itt kezeljük a konkrét ételeket és italokat.

- **Teljes kínálat lekérése (GET):** Visszaadja a teljes étlapot, ami főleg a szabadszavas keresőfunkciókhoz hasznos a tableteken.
- **Termék menedzsment és Napi ajánlat (POST, PUT, DELETE):** Ezen a végponton keresztül tudja az üzletvezető frissíteni az árakat vagy a termékfotók URL-jeit. Itt kapott helyet a **Featured** (Kiemelt) funkció is: ha egy ételt PUT kéréssel kiemeltre állítanak, az a frontend kezdőképernyőjén azonnal "Napi ajánlatként" fog pompázni.

5. RendelesController – A tranzakciók "fejléce"

Ez a kontroller fogja össze a vendég rendelésének alapadatait, úgymint az időpontot, az asztalt és a pillanatnyi státuszt.

- **Rendelés leadása (POST):** Létrehoz egy új tranzakciót az adatbázisban, és azonnal visszaadja a generált egyedi rendelés-azonosítót, ami elengedhetetlen a kosár tartalmának (a tételeknek) a rögzítéséhez.
- **Státuszok frissítése (PUT):** Ezen a végponton keresztül kommunikál a konyha a vendégtérrel. Ha a szakács a "Leadva" státuszt "Készül"-re állítja, a vendég tabletje egy frissítés után már az új állapotot mutatja.
- **Komplex lekérdezés (GET /{id}?ext=true):** Egyetlen masszív hívással, egyszerre három tábla összekapcsolásával visszaadja a rendelés fejlécét, a pincér adatait, az asztalt, és a kosár összes tételét.

6. RendelesTételController – A kosár tartalma

Míg az előző kontroller a boríték volt, ez maga a levél. Ez felel a kiválasztott ételekért és mennyiségekért.

- **Tételek felvitele (POST):** A rendelés-azonosító ismeretében itt küldi be a kliens, hogy miből hány darabot kér a vendég.
- **Módosítás és Törlés (PUT, DELETE):** Ha a vendég meggondolja magát (pl. mégis kér jeget az italba, vagy törölni akar egy tételt), a személyzet ezen a végponton tudja utólag szerkeszteni a kosarat, amíg a konyha el nem kezdte a főzést.

7. PincerHivasController – Asszisztencia a vendégeknek

A pincérhívó gomb backend logikája.

- **Hívás kezdeményezése (POST):** A rendszer itt nagyon rugalmas. Ha a kliens nem küld pontos időbélyeget vagy státuszt a kérdésben, a szerver intelligensen behelyettesíti az aktuális időt (DateTime.Now) és beállítja a "Függőben" státuszt a mentés előtt.
- **Hívások kezelése (GET és PUT):** A személyzet lekérdezi az aktív hívásokat (kiterjesztett paraméterrel az asztalszámot is megkapva), majd miután kimentek az asztalhoz, a PUT végponton keresztül "Teljesítve" státuszúra állítják, így az lekerül a várólistáról.

8. ErtekelesController – Minőségbiztosítás

A tranzakció lezárása utáni visszajelzések kezelése.

- **Értékelés mentése (POST):** A fizetés után a vendég által adott 1-5 csillagos értékelést és a szöveges véleményt összeköti a rendelés azonosítójával és elmenti.
- **Moderálás (DELETE):** A személyzet számára fenntartott végpont, amivel a véletlenül leadott, vagy sértő értékeléseket tudják eltávolítani az adatbázisból.

9. PincerController – HR adminisztráció

A dolgozói törzsadatok egyszerű felülete.

- **Nyilvántartás (GET, POST, PUT, DELETE):** Listázza az elérhető felszolgálókat (ami a rendelések kiosztásához kell), illetve a vezető ezen a végponton tud új dolgozókat felvenni, vagy műszakváltáskor az aktív státuszukat (Munka boolean érték) módosítani.

XI. A Backend Unit Tesztelése és Minőségbiztosítása

A rendszer stabilitását kiterjedt, automatizált Unit tesztekkel garantáljuk. A tesztelés célja, hogy egy-egy jövőbeli módosítás ne rontsa el a már működő üzleti logikát (regresszió elkerülése).

A környezetet az **ASP.NET Core** beépített eszköztárára és az **MSTest** keretrendszerre építettük, .NET 8.0 alatt. A tesztek futását metódusszintű párhuzamosítással gyorsítottuk fel. A legnagyobb kihívást az adatbázis kezelése jelentette. Hogy ne piszkoljuk össze az éles MySQL adatbázist, egy memóriában futó **In-Memory SQLite** megoldást alkalmaztunk. A DbContextHelper osztályunk minden egyes teszt elindulásakor generál egy friss, tiszta adatbázist a memóriában, feltölti alapvető teszt-adatokkal, majd a teszt végén megsemmisíti azt. A kódban az **AAA (Arrange-Act-Assert)** tervezési mintát követtük a tesztek strukturálására.

A Kontrollerek Tesztosztályainak Bemutatása

A kontrollerek mindegyike megkapta a saját tesztosztályát, hogy funkcióik a lehető legapróbb részletekig vizsgálhatók legyenek.

1. AsztalControllerTest

Az étterem fizikai asztalainak tesztelése során az alábbiakat vizsgáltuk:

- **Sikeres listázás:** A GetAsztalok_ReturnsOk teszt ellenőrzi, hogy a rendszer képes-e hiba nélkül visszaadni az összes memóriában lévő asztalt egy 200 OK kód kíséretében.
- **Hibakezelés (404):** A GetAsztal_InvalidId_ReturnsNotFound szándékosan egy nem létező azonosítót küld be. Bizonyítja, hogy a szerver nem omlik össze, hanem kezeli a kivételt.

- **Validációs hibák (400):** A `PostAsztal_InvalidModelState` egy hiányos modellt küld be (pl. férőhely nélkül). A teszt igazolja, hogy az API elutasítja a csonka adatokat egy `BadRequest` kíséretében.

2. *TermekControllerTest*

Az étlap komplex logikájának ellenőrzése:

- **Relációk tesztelése:** A `GetTermek_WithExtension` teszt azt igazolja, hogy az `Eager Loading` működik, és a termék mellé a rendszer gond nélkül áthúzza az ételtípus (kategória) nevét is.
- **Napi ajánlat funkció:** Ez a teszt egy `PUT` hívással megpróbálja átbillenteni a termék *Featured* státuszát igazra, majd egy lekérdezéssel ellenőrzi az `Assert` fázisban, hogy a módosítás fizikailag is végbement-e.
- **Lánc Életciklus (TestAll_ChainTest):** A legbiztosabb teszt. Egy folyamatban hoz létre egy új ételt, lekérdezi, módosítja az árát, majd véglegesen törli, minden ponton validálva az API válaszait.

3. *RendelesControllerTest*

A tranzakciós motor, a rendszer legkritikusabb részének vizsgálata:

- **Fejléc létrehozása:** A `PostRendeles_CreatesHeader` teszt ellenőrzi, hogy sikeres mentés után a backend visszaadja-e a generált `RendelesId`-t, ami a kosár összeállításához kell.
- **Státuszváltások:** A `PutRendeles_UpdatesStatus` a konyhai munkafolyamatot szimulálja, "Leadva" állapotból "Készül"-re állítva a rendelést, ellenőrizve az adatbázis frissülését.
- **Mély lekérdezések:** A `GetRendeles_WithExtension` teszt azt figyeli, hogy az asztal, a pincér és a rendelési tételek mind hibátlanul megérkeznek-e egyetlen masszív JSON csomagban.

4. *EtelTipusControllerTest*

Az étlap felépítésének adatintegritási vizsgálata:

- **Lánc lekérdezés:** A `GetEtelTipus_WithExtension_ReturnsProducts` teszt igazolja, hogy ha lekérünk egy kategóriát, a szerver tényleg kilistázza-e a hozzá tartozó termékeket a külső kulcsok (Foreign Keys) alapján.
- **Biztonságos Törlés:** A `DeleteEtelTipus_ValidId` végigzongorázza a törlési metódust, és ellenőrzi a 204 NoContent választ.

5. *PincerHivasControllerTest*

A vendégtéri asszisztencia automatizmusainak tesztelése:

- **Okos kiegészítések:** A `PostHivas_AutoFillsDefaults` teszt az API intelligenciáját vizsgálja. Szándékosan időbélyeg és státusz nélkül küldünk be egy hívást, majd ellenőrizzük, hogy a kontroller beállította-e magától a szerveridőt és a "Függőben" státuszt.
- **Folyamat lezárása:** A `PutHivas_CompletesCall` szimulálja a felszolgáló gombnyomását, mikor a hívást "Teljesítve" állapotba teszi, ami lekerül az aktív képernyőről.

6. *ErtekelesControllerTest*

A visszajelzések adatbázis-szintű ellenőrzése:

- **Relációk mentése:** A `PostErtekeles_SavesCorrectly` teszt fiktív értékelést ad le, és azt figyeli, hogy a megadott pontszám pontosan ahhoz a rendelés ID-hoz kapcsolódik-e az adatbázisban, amihez beküldtük.
- **Törlési jogok:** A `DeleteErtekeles_ReturnsNoContent` a moderálási funkciót teszteli, biztosítva, hogy a bejegyzés ténylegesen törlődik a memóriából.

XII. Manuális Tesztelési Összegzés (Felhasználói Tesztek)

Az automatizált Unit tesztek mellett elengedhetetlen volt a Gusto Bistro rendszerének manuális, felhasználói szemszögből történő (End-to-End) tesztelése is. Ennek során a szimuláltuk az étterem mindennapi működését, hogy megbizonyosodjunk a frontend (felhasználói felület) és a backend hibátlan együttműködéséről, valamint a felhasználói élmény (UX) minőségéről.

A tesztelést egy asztali számítógépen futó tesztszerveren (<https://kioskapi.lexonn.hu>) és egy laptopon végeztük el. Az alábbi összegzés az üzleti folyamatok tesztéseit (Test Cases) és azok eredményeit tartalmazza.

Ez a gyakorlatban tableteken történik ahogy az alábbi teszteken láthatjuk:

Első teszt:

Tesztelt funkció: Bejelentkezés vendégként (Guest jogosultság)

Modul: Autentikáció és Biztonság

Előfeltétel: A szerver és az adatbázis fut, a tablet a bejelentkező képernyőn áll.

Lépések:

1. A legördülő menüből kiválasztjuk az "Asztal 1" opciót.
2. Megadjuk az érvényes mesterjelszót.
3. Rákattintunk a "Belépés" gombra.

Elvárt eredmény: A rendszer a háttérben legenerálja a JWT token, azonosítja a vendég jogosultságot, és sikeresen átirányít a fő étlap képernyőre.

Tényleges eredmény: A rendszer azonnal, hiba nélkül átirányított a megfelelő felületre.

Státusz: SIKERES

Második teszt:

Tesztelt funkció: Hibás jelszó megadása

Modul: Autentikáció és Biztonság

Előfeltétel: A tablet a bejelentkező képernyőn áll.

Lépések:

1. A legördülő menüből kiválasztjuk a "Konyha" opciót.

2. Szándékosan hibás jelszót gépelünk be.
3. Rákattintunk a "Belépés" gombra.

Elvárt eredmény: A rendszer megtagadja a belépést, és egy egyértelmű, magyar nyelvű hibaüzenetet jelenít meg a képernyőn (pl. "Helytelen jelszó lett megadva ehhez: 'Konyha' !").

Tényleges eredmény: A belépés blokkolásra került, a hibaüzenet olvashatóan megjelent.

Státusz: SIKERES

Harmadik teszt:

Tesztelt funkció: Termék kosárba tétele és a kosár frissülése

Modul: Étlap és Kosárkezelés

Előfeltétel: A vendég be van jelentkezve, az étlap nézet aktív.

Lépések:

1. Kiválasztjuk a "Pizzák" kategóriát.
2. Rákattintunk a "Margherita pizza" alatti "Kosárba" gombra.
3. Ellenőrizzük a jobb oldali kosár sávot.

Elvárt eredmény: A termék azonnal, oldalfrissítés nélkül megjelenik a kosárban. Az alsó sávban a fizetendő végösszeg automatikusan frissül a pizza árával.

Tényleges eredmény: A React állapotkezelője azonnal frissítette a kosarat és az árat.

Státusz: SIKERES

Negyedik teszt:

Tesztelt funkció: Kosár tartalmának módosítása és törlése

Modul: Étlap és Kosárkezelés

Előfeltétel: A kosárban már szerepel 1 db Margherita pizza.

Lépések:

1. A kosárban a pizza melletti "+" gombra kattintunk.
2. Ezután a mellette lévő "Kuka" (Törlés) ikonra kattintunk.

Elvárt eredmény: A "+" gomb hatására a mennyiség 2-re, a végösszeg a duplájára nő. A törlés ikonra kattintva a termék teljesen eltűnik a kosárból, a végösszeg pedig visszaáll 0 Forintra.

Tényleges eredmény: A mennyiség növelése és a tétel törlése is a vártak megfelelően működött.

Státusz: SIKERES

Ötödik teszt:

Tesztelt funkció: Rendelés véglegesítése és leadása

Modul: Rendelési folyamat (Tranzakció)

Előfeltétel: A kosárban szerepel egy véglegesített étellista.

Lépések:

1. Rákattintunk a "Rendelés leadása" gombra.
2. Kiválasztjuk a Készpénzes fizetési módot a felugró ablakban.

Elvárt eredmény: A kosár tartalma elküldésre kerül az adatbázisba (backend). A tablet képernyője átvált a folyamatjelző nézetre, ahol megjelenik a "Leadva" státusz. A kosár kiürül.

Tényleges eredmény: Az Fetch sikeresen elküldte a kérést, az adatbázisban létrejött a rekord, a folyamatjelző megjelent.

Státusz: SIKERES

Hatodik teszt:

Tesztelt funkció: Konyhai státuszváltás szinkronizációja

Modul: Konyhai felület

Előfeltétel: Nyitva van egy konyhai (Személyzet) tablet, és egy vendég (Asztal) tablet.
Az asztalról érkezett egy új rendelés.

Lépések:

1. A konyhai tableten a szakács rákattint a beérkezett rendelésnél a "Készítés megkezdése" gombra.
2. Ellenőrizzük a vendég tabletjét.

Elvárt eredmény: A konyhai kártya státusza frissül az adatbázisban, majd a vendég tabletjén lévő folyamatjelző is azonnal, magától átvált a "Készül" állapotra.

Tényleges eredmény: A státuszváltás sikeres volt, a vendég azonnali vizuális visszajelzést kapott.

Státusz: SIKERES

Hetedik teszt:

Tesztelt funkció: Pincér hívása gomb működése

Modul: Személyzeti kommunikáció

Előfeltétel: A vendég be van jelentkezve. A pincérek felülete egy másik gépen aktív.

Lépések:

1. A vendég rákattint a képernyő sarkában lévő "Pincér hívása" ikonra.
2. Megnézzük a pincérek összesítő listáját.

Elvárt eredmény: A vendég tabletjén a gomb színt vált (jelezve, hogy a hívás elment). A pincérek felületén azonnal megjelenik egy új bejegyzés az asztal számával és a pontos időponttal, "Függőben" státusszal.

Tényleges eredmény: A hívás rögzítése és a személyzeti listán való megjelenése hibátlanul lefutott.

Státusz: SIKERES

Nyolcadik teszt:

Tesztelt funkció: Értékelés leadása a fogyasztás végén

Modul: Minőségbiztosítás és Statisztika

Előfeltétel: A rendelés státusza "Kész", a fizetés megtörtént.

Lépések:

1. A felugró értékelő ablakban rákattintunk az 5 csillagra.
2. Beírjuk a szövegdobozba, hogy "Kiváló volt a pizza!".
3. Rákattintunk a "Küldés" gombra.

Elvárt eredmény: Az értékelés ablak eltűnik, a rendszer megköszöni a visszajelzést. Az adatok (pontszám, szöveg, rendelés ID) elmentésre kerülnek az adatbázis "ertekeles" táblájába.

Tényleges eredmény: A felhasználói felület megfelelően reagált, az adatbázisban az új rekord ellenőrzése sikeres volt.

Státusz: SIKERES

XIII. A Frontend (Kliensoldal) felépítése és működése

Amikor elkezdünk tervezni a Gusto Bistro felhasználói felületét, a legfőbb szempont a sebesség és az átláthatóság volt. Mivel a szoftver éttermi asztalokon, tableteken fog futni, nem engedhettük meg, hogy a vendégeknek várniuk kelljen az oldalak betöltésére. Ezért döntöttünk a React keretrendszer mellett, amivel egy úgynevezett "Single Page Application-t" hoztunk létre. Ez azt jelenti, hogy a használat során az oldal nem töltődik újra, hanem a képernyő elemei villámgyorsan cserélődnek a háttérben.

1. Components és pages különbsége

Sok React projektben hajlamosak túlbonyolítani a mappaszerkezetet, nekünk viszont a tiszta és logikus felépítés volt a feladatunk, így tehát egyben célunk is. A projektünket mindössze két fő mappára bontottuk, ami megkönnyítette a közös munkát:

- **Komponensek (components):** Ide kerültek azok a komponensek, amikből a felület építkezik. Ilyen például egy gomb, a kosár egy sora, vagy az a kártya, ami az étlapon egy-egy konkrét ételt jelenít meg. Mivel ezeket külön fájlként kezeltük, bárhol újra tudtuk őket használni a programban.
- **Oldalak (pages):** Itt találhatóak maguk a teljes képernyős nézetek, mint például a Bejelentkező oldal, a fő Étlap nézet, vagy a Konyhai feladatkezelő tábla. Ezek az oldalak a components mappában lévő kisebb elemekből állnak össze.

2. A projektünkhöz használt képek helye

Érdekessége a frontendünknek, hogy nem tartalmaz külön mappát a képek vagy ikonok számára (nincs assets mappa). Ennek teljesítménybeli oka van: szerettük volna, ha a kliensoldali programunk (a frontend) mérete a lehető legkisebb marad, hogy a tabletek böngészője pillanatok alatt be tudja tölteni. Ehelyett a termékek fotóit külsős, felhőalapú képmegosztó oldalon (Imgur) tároljuk. A mi adatbázisunk a termék táblában csak a képek weblinkjét (URL-jét) menti el. Amikor a vendég megnyitja az étlapot, a tablet egyszerűen letölti a képeket a külső szerverről, így a mi saját alkalmazásunk rendkívül könnyű és gyors maradt.

3. A szerver elérése

A hálózati kommunikáció kialakításánál nem hoztunk létre külön, központosított szolgáltatás-réteget (services) a szerverhívásoknak. Ehelyett az API hívásokat (az adatok lekérését vagy elküldését) minden esetben lokálisan, közvetlenül azon a komponensen belül valósítottuk meg, amelyiknek szüksége van rá. Ha például a "Konyha" oldalnak le kell kérnie a rendeléseket, az ehhez szükséges adatlekérés (Fetch hívás) magában a konyha komponens kódjában található. Ez a megközelítés azért zseniális, mert ha bármikor módosítani akarunk egy funkciót, pontosan látjuk egyetlen fájlban belül, hogy milyen adatok érkeznek a szerverről, és azok hogyan jelennek meg a képernyőn. A biztonsági azonosítót (JWT token) is maguk a komponensek olvassák ki a böngésző memóriájából, és csatolják a kérések mellé.

4. A kosár tartalmának kezelése

Mivel az adatkéréseket maguk a komponensek intézik, az adatok tárolását is a React beépített helyi állapotkezelőivel (useState és useEffect) oldottuk meg. A kosár működése például úgy épül fel, hogy a fő "Étlap" oldal tartja nyilván egy virtuális jegyzetömbben, hogy a vendég miket válogatott össze. Amikor a vendég rákattint egy termékre, a komponens jelez a főoldalnak, ami frissíti ezt a memóriát. Ha a termék már a listán van, csak a darabszámát növeli, ha nincs, hozzáadja új tételként. Ez az adat egészen addig él, amíg a rendelés gomb megnyomásával el nem küldjük a szervernek.

5. Gyakorlatbeli működése

A felület fekvő tájolásra lett optimalizálva. A gombokat szándékosan nagyra méreteztük, hogy egy beszélgető vendég vagy egy sietős pincér is kényelmesen, félrekattintás nélkül tudja használni. A dizájnban erős vizuális kontrasztokat alkalmaztunk: az olyan kritikus gombok, mint a "Fizetés" vagy a "Pincér hívása" élénk, domináns színeket kaptak, miközben az allergének vagy az ételleírások finomabb, elegánsabb formában jelennek meg, biztosítva a rendszer átláthatóságát. Maga a konyhai oldal is olyan színeket kapott, ami az éttermekben lévő standard szabályoknak megfelel, így nagyobb pára, gőz esetén is tökéletesen láthatóak lesznek a kijelző képernyőjén megjelenő adatok.

XIV. Telepítési és Fejlesztői Útmutató

Ahhoz, hogy a Gusto Bistro éttermi rendelési rendszer a fejlesztői környezetből egy másik számítógépre kerüljön, pontosan követni kell a telepítési lépéseket. Mivel a szoftverünk egy modern, háromrétegű architektúra (Adatbázis, Backend API, Frontend Kliens), mindhárom réteget külön-külön kell inicializálni és elindítani.

1. Rendszerkövetelmények és szükséges szoftverek (Előfeltételek)

A rendszer futtatásához a gazdagépen (szerveren) az alábbi szoftveres környezetnek kell telepítve lennie:

- **XAMPP Control Panel:** A MySQL adatbázis és az Apache webszerver futtatásához.

- **Node.js:** A React alapú frontend keretrendszer csomagjainak letöltéséhez és a fejlesztői szerver futtatásához (tartalmazza az npm csomagkezelőt).
- **Visual Studio 2022 (vagy újabb):** A C# backend projekt megnyitásához, fordításához és futtatásához.
- **.NET 8.0 SDK:** A Microsoft hivatalos fejlesztői környezete, amely a Web API működéséhez elengedhetetlen.

2. Az Adatbázis (MySQL) beüzemelése

A szoftver "memóriájának" felélesztése az első és legfontosabb lépés, hiszen enélkül a backend nem tud hova csatlakozni.

1. Lépés: Nyissuk meg a XAMPP Control Panel alkalmazást rendszergazdai jogosultsággal.
2. Lépés: Indítsuk el az Apache és a MySQL modulokat a mellettük található "Start" gombok megnyomásával. Ha a modulok nevei zöldre váltanak, a szerverek sikeresen futnak.
3. Lépés: Nyissunk meg egy tetszőleges webböngészőt (pl. Google Chrome), és a címsorba gépeljük be a "localhost/phpmyadmin" címet.
4. Lépés: A bal oldali panelen hozzunk létre egy új adatbázist "rendelotablet" néven. Fontos, hogy az illesztés (Collation) beállításnál az "utf8_general_ci" (vagy utf8mb4) kódolást válasszuk, hogy a magyar ékezetes karakterek (pl. a termékneveknél) helyesen jelenjenek meg.
5. Lépés: Kattintsunk a felső menüsorban az "Importálás" fülre.
6. Lépés: A "Fájl kiválasztása" gombbal tallózzuk ki a projektünk mappájából a "rendelotablet.sql" nevű fájlt, majd a lap alján nyomjuk meg az "Indítás" gombot.
7. Lépés: Sikeres importálás után a bal oldali sávban megjelenik a 8 darab feltöltött tábla (asztal, termék, rendeles, stb.), bennük a fejlesztés során rögzített tesztadatokkal.

3. A Szerveroldal (Backend Web API) elindítása

Miután az adatbázis fut, el kell indítanunk a C# API-t, amely kiszolgálja majd a tabletek kéréseit.

1. Lépés: Navigáljunk a backend projektünk főkönyvtárába, és nyissuk meg a ".sln" (Solution) kiterjesztésű fájlt a Visual Studio segítségével.
2. Lépés: A Visual Studio a megnyitás után automatikusan ellenőrzi a hivatkozásokat. Várjuk meg, amíg a NuGet csomagkezelő letölti a szükséges függőségeket (pl. Entity Framework Core, JWT Bearer csomagok).
3. Lépés: A Megoldáskezelőben (Solution Explorer) keressük meg és nyissuk meg az "appsettings.json" fájlt.
4. Lépés: Ellenőrizzük a "ConnectionStrings" blokkot. Itt kell lennie az adatbázis elérési útjának. Alapértelmezett XAMPP telepítés esetén a felhasználónév "root", a jelszó pedig üres.
5. Lépés: Indítsuk el a projektet a felső menüsávban található zöld "Play" (Indítás) gombbal, vagy az F5 billentyű lenyomásával.
6. Lépés: A Visual Studio lefordítja a kódot, és megnyit egy parancssori ablakot, valamint a böngészőben a Swagger API dokumentációs felületet. A backend innentől kezdve aktívan "hallgatózik" a megadott localhost porton (pl. <https://localhost:7123>).

4. A Kliensoldal (React Frontend) elindítása

A backend futása közben már csak a felhasználói felületet kell életre keltetnünk.

1. Lépés: Nyissuk meg a számítógép fájlkezelőjében a frontend React projekt mappáját.
2. Lépés: Kattintsunk a mappa címsorába, írjuk be a "cmd" parancsot, és nyomjunk Entert. Ez megnyit egy parancssori ablakot pontosan abban a mappában.
3. Lépés: A hiányzó Node csomagok letöltéséhez adjuk ki az "npm install" parancsot. (Ez a lépés csak az legelső indításkor kötelező, vagy ha egy fejlesztőtárs új csomagot adott a projekthez).
4. Lépés: A letöltés befejezése után (amikor létrejött a node_modules mappa), adjuk ki az "npm start" parancsot a fejlesztői szerver elindításához.
5. Lépés: A folyamat végén a rendszer automatikusan megnyitja a böngészőben a "localhost:3000" címet, ahol megjelenik a Gusto Bistro bejelentkező képernyője.

Ha minden lépést pontosan követtünk, a frontend sikeresen kapcsolódik a futó backendhez, a backend pedig az XAMPP adatbázisához. A rendszer ezzel a tesztelésre és a használatra kész állapotba került!

XV. Felhasználói Kézikönyv és Útmutató

A Gusto Bistro szoftverrendszer tervezésekor kiemelt figyelmet fordítottunk az ergonómiára és a könnyű használhatóságra. Mivel a rendszert a vendégek előzetes betanítás nélkül használják, a felületnek intuitívnek és egyértelműnek kell lennie. Az alábbi kézikönyv lépésről lépésre bemutatja a rendszer funkcióit a két fő felhasználói csoport: a vendégek és a személyzet szemszögéből. Minden felhasználónak (Konyha, Asztal) azonos a jelszava, jelen esetben '1234' ez a kódban módosítható.

1. Vendég felület (Asztali táblagép használata)

A vendégtérben elhelyezett tableteken a szoftver "Guest" (Vendég) jogosultsággal fut, így a vásárlók csak a saját asztalukhoz tartozó funkciókhoz férnek hozzá.

1. Kezdőképernyő és Asztal kiválasztása: A tablet bekapcsolása után a kezdőképernyő fogadja a felhasználót. Az étterem nyitásakor a személyzet kiválasztja a legördülő menüből az adott asztal számát (pl. "Asztal 1"), és a megfelelő jelszóval aktiválja a gépet. Ezt követően a képernyőn megjelenik a Gusto Bistro üdvözlőfelülete és az aktuális Napi Ajánlat (Featured termék).

2. A digitális étlap böngészése: A "Rendelés megkezdése" gombra kattintva a vendég a fő étlapra jut.

- A képernyő bal oldalán vagy tetején találhatók a főkategóriák (Pizzák, Levesek, Italok stb.). Egy kategóriára kattintva a felület azonnal, betöltési idő nélkül listázza az oda tartozó ételeket.
- Minden ételkártyán jól látható a termék neve, egy ínycsiklandó fotó, az ár forintban, valamint – egy kis ikon vagy szöveg formájában – az allergén információk (pl. glutén, laktóz), segítve az ételérzékenységgel rendelkező vendégeket.

3. Kosárba tétel és a Kosár kezelése Amikor a vendég kiválaszt egy szimpatikus ételt, a "Kosárba" gomb megnyomásával az bekerül a rendelési listába.

- A Kosár tartalma a képernyő jobb oldalán (vagy egy külön felugró ablakban) folyamatosan nyomon követhető.

- A vendég a "+" és "-" gombok segítségével bármikor módosíthatja az adagok számát, vagy a "Törlés" (Kuka) ikonnal kivehet egy tételt a listából.
- A rendszer az alul található sávban automatikusan és valós időben számolja a fizetendő végösszeget.

4. Rendelés leadása és Nyomon követés Miután a vendég összeállította a rendelését, a "Rendelés leadása" gombbal véglegesítheti azt. Ekkor a rendszer a kosár tartalmát elküldi a konyhára. A rendelés leadása után a képernyőn megjelenik egy interaktív folyamatjelző (Status Bar). Itt a vendég valós időben láthatja, hogy mi történik az ételével:

- **Leadva:** A rendelés beérkezett a rendszerbe.
- **Készül:** A szakácsok elkezdtek az étel elkészítését.
- **Tálalás:** Az étel elkészült, a pincér hamarosan kihozza az asztalhoz.

5. Pincérhívás és Fizetés A szoftver megszünteti a hosszas várakozást és a pincérek keresgélését. A képernyő sarkában található "Pincér hívása" (Csengő vagy Kérdőjel) ikonra kattintva a vendég azonnal jelezni tud a személyzetnek. Ugyanitt található a "Fizetés" gomb is, amellyel a fogyasztás végén jelezhető a fizetési szándék.

6. Értékelés A fizetés befejezése után a rendszer egy rövid visszajelző ablakot dob fel. Itt a vendégek 1-től 5-ig terjedő csillagos skálán értékelhetik az ételeket és a kiszolgálást, valamint szöveges megjegyzést is hagyhatnak. Ez hatalmas segítség az étterem vezetőségének a minőség fenntartásában.

2. Személyzeti felület (Konyha és Felszolgálók)

A konyhában elhelyezett kijelzők, illetve a felszolgálók mobil eszközei "Staff" (Személyzet) jogosultsággal jelentkeznak be a rendszerbe. Ez a felület szigorúan a munkavégzést és a folyamatok menedzselését szolgálja.

1. Konyhai Feladatkezelő Tábla: A konyhai nézet egy Kanban-stílusú, sötét témájú felület, amely nem vakítja el a szakácsokat a munkakörnyezetben.

- Amint egy vendég lead egy rendelést, az azonnal, oldalfrissítés nélkül megjelenik a konyhai kijelző "Új rendelések" oszlopában egy kártya formájában.

- A kártya fejlécén hatalmas betűkkel szerepel az Asztal száma és a rendelés pontos ideje.
- Alatta tételesen olvashatók a kosárba tett ételek és azok mennyisége.

2. Státuszok módosítása a konyhán: A szakácsok feladata, hogy a kártyákon található gombokkal jelezzék a munkafolyamat aktuális állapotát.

- Amikor elkezdik sütni a pizzát, ráböknek a kártyán a "Készítés megkezdése" gombra. Ez a rendszerben a státuszt "Készül"-re váltja (amit a vendég is lát a saját asztalánál).
- Amikor az étel elkészült, a "Kész / Tálalás" gomb megnyomásával a kártya átkerül a kész ételek oszlopába, és a rendszer egy értesítést küld a pincéreknek, hogy vihetik az ételt az asztalhoz.

3. Felszolgálói nézet és Hívások kezelése: A pincérek egy speciális összesítő nézetet látnak a saját készülékeiken.

- A képernyőn listázva jelennek meg az aktív "Pincérhívások". A lista mutatja az asztal számát, a kérés idejét, és a kérés típusát (pl. Fizetés készpénzzel, vagy Általános segítségkérés).
- Miután a pincér odament az asztalhoz és teljesítette a kérést (például odaadta a számlát és átvette a pénzt), a saját tabletjén rányom a "Nyugtázva / Teljesítve" gombra. Ekkor a kérés eltűnik a várakozó listából, fenntartva a rendszer átláthatóságát.

XVI. Kommunikációs eszközök és kapcsolattartás:

Verziókezelés - GitHub

Fájlok, dokumentumok megosztása egymással - Google Drive

Megbeszélések - Discord

Sima beszélgetések - Messenger

Órai közös munka

XVII. Segédoldalak, források:

Ételekről képek generálása: <https://gemini.google.com/app?hl=hu>

További képek: <https://pixabay.com/hu/photos/>

Betűtípus: <https://fonts.google.com/>

Színek: <https://coolors.co/?home>

Ikonok: <https://fonts.google.com/icons>

Imgur, a képeket ide töltöttük fel: <https://imgur.com/>

Egyéb : <https://www.w3schools.com/>

StackOverflow: <https://stackoverflow.com/questions>

Backend - Frontend összekötése: Órán tanultak alapján

XVIII. Továbbfejlesztési lehetőségek

A projektnek több továbbfejlesztési lehetősége van.

Bankkártya-terminál a tablet mellé:

Még fejlettebb fizetés érdekében bankkártya-terminállal lehetővé lehetne tenni a bankkártyás fizetést. Így a pincérnek csak a készpénzes fizetés esetén kell az asztalhoz menni,

Statisztikai adatok:

A rendszert statisztikai adatok tárolásával lehetne még fejleszteni. A konyha visszajelzést kap, hogy melyik termék volt a legnépszerűbb. Ezeket rangsorolva így könnyen áttekinthető, hogy melyik étel, ital volt fogyasztva a vendégek által a leggyakrabban.

QR kód használata:

A vendég a tableten megjelenő QR kód beolvasásával, a saját készülékén egy olyan alkalmazást kap mint ami a tableten megtalálható. Kényelmesebb lehet saját készüléken kiválasztani a kívánt termékeket.

Telefonos alkalmazás:

Tervben van egy telefonos alkalmazás elkészítése is. Ami azt tenné lehetővé, hogy a vendégek saját eszközről be tudjanak regisztrálni, illetve belépni az étterem rendelési felületére. Így lehetőséget biztosítunk az éttermen kívüli rendelésre, hogy a vendég mire az étterembe ér kész legyen a rendelése és ezzel is gyorsítunk a munkán, illetve a vendég is spórolhat a saját idejéből.

Pontos idő visszajelzés:

Az oldalt az étterem által kiadott étel elkészítési idővel is ki lehet majd a későbbiekben egészíteni. Ami annyit takar, hogy a vendég nem csak az alap szavakat, mint például leadva, készül stb. látja hanem időre pontosan kap visszajelzést az étel érkezéséről. Például megjelenik egy számláló, egy 10 perces időzítővel és mikorra ez az idő lejár készen van az étel.

Számlabontás:

A pincérek egyik legidőigényesebb feladata a nagyobb asztaltársaságok számlájának megbontása. A tablet felületén egy új funkcióval a vendégek már a fizetés gomb megnyomása előtt kiválaszthatnák, hogy egyben, egyenlő arányban elosztva, vagy tételesen (ki mit fogyasztott) szeretnék kifizetni a végösszeget. A rendszer ez alapján küldené a bontott blokk kérését a kasszához vagy a bankkártya-terminálhoz.

Ételek testreszabhatósága és szűrése:

Bár az allergének már most is fel vannak tüntetve, a felületet tovább lehetne fejleszteni egy dinamikus szűrőrendszerrel (pl. "Csak a gluténmentes és vegán ételek mutatása"). Emellett a kosárba tételkor a vendégek módosíthatnák az összetevőket: kérhetnének extra feltéteket felár ellenében, vagy kivehetnének nem kívánt összetevőket (pl. "hagyma nélkül"), ami egyedi megjegyzésként jelenne meg a konyhai blokkon.

Többnyelvűség és turistabarát felület:

Mivel a papíralapú étlapok cseréje drága, a digitális rendszer hatalmas előnye a többnyelvűség. A jövőben a szoftvert fel lehetne készíteni az angol, német vagy egyéb nyelvek támogatására. Egy gombnyomással a teljes felület, beleértve a termékleírásokat és az allergén-információkat is, átválthatna a kívánt nyelvre, ami turisztikailag népszerű helyeken óriási versenyelőnyt jelentene az étteremnek.

Hűségprogram:

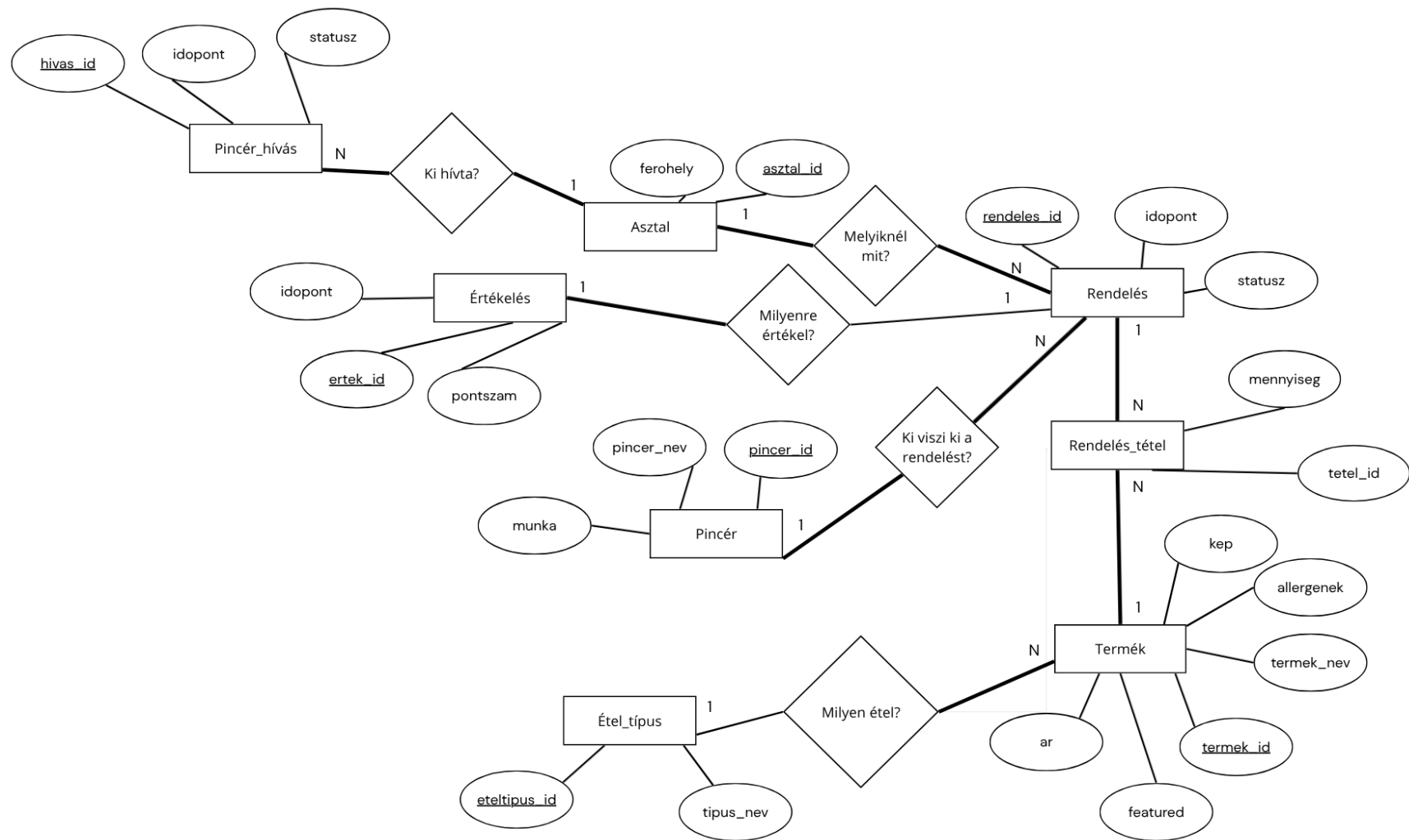
A bejelentkező rendszer (telefonos applikáció) bevezetésével párhuzamosan elindítható egy törzsvásárlói program. A vendégek a rendeléseik után pontokat gyűjthetnek, amiket később kedvezményekre válthatnak be. Természetesen ezt az étterem maga dönti el, hogy szeretné alkalmazni ezeket a hűségpontokat.

Tartalom

I.	Bevezetés	1
II.	Szoftver célja	1
III.	Célközönség:	2
IV.	Feladatok:	2
V.	Komponenseinek technikai leírása	3
	Frontend:.....	3
	Backend:	3
	Adatbázis:	4
VI.	Működésének műszaki feltételei	4
	1. Hardveres eszközök.....	4
	2. Hálózati infrastruktúra	5
VII.	Technológiai háttér:.....	5
VIII.	Használatának rövid bemutatása:	6
	Bejelentkezés	15
IX.	Adatbázis terv (ER modell) [Melléklet: 14. ábra]:	16
	Táblák és mezők:	16
	Entitások Közötti Kapcsolatok (Idegen Kulcsok)	18
X.	A Backend Kontrollerek Működése és Végpontjai	19
	1. AuthController – A rendszer biztonsági kapuja	19
	2. AsztalController – Az éttermi térkép kezelése	20
	3. EtelTipusController – A digitális étlap kategóriái.....	20
	4. TermekController – Az étlap konkrét tételei	21
	5. RendelesController – A tranzakciók "fejléce"	21
	6. RendelesTételController – A kosár tartalma	21
	7. PincerHivasController – Asszisztencia a vendégeknek.....	22
	8. ErtekelesController – Minőségbiztosítás.....	22

9. PincerController – HR adminisztráció	22
XI. A Backend Unit Tesztelése és Minőségbiztosítása	23
A Kontrollerek Tesztosztályainak Bemutatása.....	23
1. AsztalControllerTest.....	23
2. TermekControllerTest	24
3. RendelesControllerTest	24
4. EtelTipusControllerTest	24
5. PincerHivasControllerTest	25
6. ErtekelesControllerTest	25
XII. Manuális Tesztelési Összegzés (Felhasználói Tesztek)	25
XIII. A Frontend (Kliensoldal) felépítése és működése	30
1.Components és pages különbsége	31
2. A projektünkhöz használt képek helye.....	31
3. A szerver elérése.....	31
4. A kosár tartalmának kezelése	32
5. Gyakorlatbeli működése	32
XIV. Telepítési és Fejlesztői Útmutató	32
XV. Felhasználói Kézikönyv és Útmutató	35
XVI. Kommunikációs eszközök és kapcsolattartás:.....	37
XVII. Segédoldalak, források:	38
XVIII. Továbbfejlesztési lehetőségek	38

Melléklet:



14. Ábra